

Quilore Full Implementation Backlog (Plane-Ready)

This document converts the finalized Quilore product and architecture direction into a detailed, Plane-ready backlog intended for full implementation rather than a narrow MVP. Quilore is positioned as an AI fitness and nutrition coach with workout logging, nutrition tracking, meal analysis, and coaching flows, and the product direction explicitly includes manual meal entry, calorie and macro target estimation, and target tracking.[cite:2] The implementation model also assumes the locked hybrid backend architecture in which Spring Boot owns stateful product domains while FastAPI owns AI orchestration and AI-heavy service workflows.[cite:23][cite:38][cite:39]

Planning assumptions

The backlog below is written to support a production-grade implementation, not just prototype feature coverage. In agile backlog practice, large epics need progressive refinement into smaller, testable stories, and non-functional requirements such as security, performance, and reliability must be made visible in the backlog instead of being left implicit.[cite:24][cite:26][cite:27][cite:28][cite:35]

The ownership model used throughout this document is:

- **Mobile:** React Native client and device capabilities.
- **Spring Boot:** Auth, profiles, goals, workout logs, nutrition logs, plans, sync, quotas, and core domain rules.[cite:23]
- **FastAPI:** AI orchestration, prompt assembly, OCR cleanup, routing across model providers, embeddings, and food-image processing pipelines.[cite:23][cite:38]
- **Infra/Data:** PostgreSQL, pgvector, MinIO, queues, observability, CI/CD, deployment, and operational tooling.[cite:38]

Epic structure

The backlog is grouped into the following product epics so the resulting Plane project remains navigable while still covering complete implementation scope:[cite:24][cite:35]

- Onboarding and identity
- Workout logging and training journal
- Workout import and exercise understanding
- Form analysis and kinematics
- AI coaching and programming
- Nutrition goals and daily tracking
- Scanned meal flow
- Manual meal entry and food resolution

- Progress, insights, and analytics
- Notifications and engagement
- Subscription, entitlements, and billing
- Offline sync and data management
- AI orchestration and resilience
- Admin and content operations
- Security, privacy, and compliance
- Observability, QA, and release engineering

Epic 1: Onboarding and Identity

Story 1.1: User registration and authentication

- **Description:** Implement account sign-up, sign-in, sign-out, token refresh, and authenticated session bootstrap.
- **Service/Owner:** Spring Boot + Mobile
- **Dependencies:** Core user schema, auth provider decision, secure secret handling
- **Acceptance Criteria:**
 - User can create an account using supported login methods.
 - User can sign in on a clean install and remain logged in across app restarts.
 - Expired access tokens are refreshed without forcing unnecessary sign-outs.
 - Invalid credentials return a user-safe error state.
 - All auth endpoints are protected with rate limiting and structured audit logging.
[cite:23][cite:28]

Story 1.2: Profile creation and baseline fitness data capture

- **Description:** Collect age, sex, height, weight, training experience, dietary preferences, injuries, and equipment access during onboarding.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Auth, profile schema, validation rules
- **Acceptance Criteria:**
 - Onboarding form validates required fields before submission.
 - User profile is persisted and editable later in settings.
 - Baseline fields are available to downstream program generation and nutrition target calculation.
 - Injury and health fields are clearly labeled as informational and non-diagnostic.

Story 1.3: Goal setup and coaching preference capture

- **Description:** Capture user goals such as fat loss, recomposition, muscle gain, maintenance, and performance focus, along with coaching tone and scheduling preferences.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Profile data, nutrition target engine, program generation rules
- **Acceptance Criteria:**
 - User can select one primary goal and optional secondary preferences.
 - Goal selection is persisted and versioned for future recalculation.
 - Coaching settings are exposed to AI prompt assembly inputs.
 - Goal changes trigger recalculation workflows for plans and targets.

Story 1.4: Consent, disclaimer, and privacy preference capture

- **Description:** Add onboarding consent capture for AI-generated outputs, injury triage disclaimers, camera/media permissions, and privacy choices.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Legal copy, settings model, audit events
- **Acceptance Criteria:**
 - User must explicitly accept required terms before completing onboarding.
 - Medical-risk features display a clear non-diagnostic disclaimer.
 - Consent versions are stored with timestamp and app version.
 - Optional privacy permissions can be revisited from settings.

Epic 2: Workout Logging and Training Journal

Story 2.1: Exercise library and routine browsing

- **Description:** Provide searchable exercise metadata, tags, movement categories, equipment, and muscle group mappings.
- **Service/Owner:** Spring Boot + Mobile
- **Dependencies:** Seed dataset, search indexing, media assets
- **Acceptance Criteria:**
 - User can browse and search exercises by name, muscle group, and equipment.
 - Exercise detail page includes instructions, movement type, and targeted muscle groups.
 - Exercise records expose stable IDs used by logging, coaching, and form-check features.
 - Mobile cache supports offline viewing of previously loaded exercises.

Story 2.2: Manual workout session creation

- **Description:** Allow user to create a workout session manually, add exercises, and record sets, reps, load, rest, RPE, and notes.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Exercise library, workout schema, local cache
- **Acceptance Criteria:**
 - User can create and save an in-progress workout.
 - Set-level fields validate numeric ranges and optional metadata.
 - Editing a set updates totals and summaries immediately.
 - Saved workouts persist offline first and sync when connectivity returns.

Story 2.3: Continuous voice workout logging

- **Description:** Implement continuous on-device voice capture for set logging with transcript confirmation UX.
- **Service/Owner:** Mobile + FastAPI + Spring Boot
- **Dependencies:** Speech engine integration, AI prompt assembly, workout write API
- **Acceptance Criteria:**
 - User can start and stop voice logging from workout mode.
 - Partial transcript is visible during capture.
 - Structured workout events extracted from speech are shown for confirmation or correction.
 - Confirmed logs are persisted to the correct workout session with full audit metadata.
 - Failed parsing returns a recoverable fallback that lets the user edit manually.

Story 2.4: Workout history and timeline view

- **Description:** Build workout history with filters, session summaries, and drill-down into prior sets and notes.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Workout persistence, query endpoints, pagination
- **Acceptance Criteria:**
 - User can view workouts by date range and routine type.
 - Session list supports pagination and pull-to-refresh.
 - Individual workout detail shows set-by-set history.
 - History queries return consistent ordering and accurate totals.

Story 2.5: Progression rules and personal records engine

- **Description:** Track PRs, rep bests, volume records, and progression suggestions from historical logs.
- **Service/Owner:** Spring Boot
- **Dependencies:** Workout history, analytics jobs, exercise identity consistency
- **Acceptance Criteria:**
 - Personal records are calculated deterministically from stored sessions.
 - User sees when a new PR is set.
 - Progression logic can suggest next-session load or rep targets.
 - PR and progression calculations are rerunnable after backfilled edits.

Epic 3: Workout Import and Exercise Understanding

Story 3.1: Routine import from pasted text

- **Description:** Parse pasted training plans into structured routines and unresolved items for user review.
- **Service/Owner:** Mobile + FastAPI + Spring Boot
- **Dependencies:** Parser logic, exercise resolution service, routine schema
- **Acceptance Criteria:**
 - User can paste freeform routine text into an import screen.
 - Parsed exercises, sets, reps, and notes are previewed before save.
 - Unknown exercise names are flagged for manual resolution.
 - Approved imports create reusable routines in the user account.

Story 3.2: OCR import for workout screenshots and photos

- **Description:** Extract routine text from images and convert it into a structured routine draft.
- **Service/Owner:** Mobile + FastAPI
- **Dependencies:** OCR provider, image upload pipeline, parser
- **Acceptance Criteria:**
 - User can upload or capture a routine image.
 - OCR output is normalized and shown as editable text before parsing.
 - Low-confidence lines are highlighted for review.
 - Successful import produces the same routine draft structure as pasted text import.

Story 3.3: Video-based exercise import and rep segmentation

- **Description:** Support video import for exercise understanding, rep segmentation, and candidate exercise identification.
- **Service/Owner:** Mobile + FastAPI
- **Dependencies:** Video frame extraction, pose inference pipeline, exercise templates
- **Acceptance Criteria:**
 - User can upload a supported video clip.
 - System extracts frames and identifies exercise candidates with confidence scores.
 - Rep segments are produced as reviewable intervals.
 - Unsupported or low-confidence videos fail gracefully with clear guidance.

Story 3.4: Exercise alias and normalization service

- **Description:** Normalize imported exercise names against a canonical exercise dictionary.
- **Service/Owner:** Spring Boot + FastAPI
- **Dependencies:** Exercise alias table, search service, admin curation tools
- **Acceptance Criteria:**
 - Common aliases map to one canonical exercise ID.
 - Resolver returns confidence and candidate matches.
 - User corrections can feed admin review queues.
 - Imported routines persist canonical IDs rather than raw text labels.

Epic 4: Form Analysis and Kinematics

Story 4.1: On-device pose capture integration

- **Description:** Integrate pose estimation for supported exercises in live capture and recorded video workflows.
- **Service/Owner:** Mobile
- **Dependencies:** Camera pipeline, supported device matrix, inference model packaging
- **Acceptance Criteria:**
 - Supported devices can capture pose landmarks during exercise recording.
 - Pose inference meets a defined minimum frame-processing threshold on supported hardware.
 - Unsupported devices receive a fallback path without blocking the rest of the workout flow.
 - Landmark streams are available to downstream angle and rep-analysis modules.

Story 4.2: Joint angle and posture deviation computation

- **Description:** Compute joint angles, bar path proxies, symmetry markers, and posture deviations from landmark data.
- **Service/Owner:** FastAPI
- **Dependencies:** Pose landmarks, exercise templates, normalization logic
- **Acceptance Criteria:**
 - Angle calculations are deterministic for the same landmark sequence.
 - Exercise-specific metrics can be configured per movement.
 - Low-confidence landmarks are excluded or flagged.
 - Computed metrics are stored with versioned algorithm metadata.

Story 4.3: Rep phase detection and timing normalization

- **Description:** Detect rep boundaries and normalize timing differences against movement templates.
- **Service/Owner:** FastAPI
- **Dependencies:** Pose landmarks, rep segmentation, exercise templates
- **Acceptance Criteria:**
 - Concentric and eccentric phases are identified for supported movements.
 - Multiple reps within a set are segmented accurately enough for downstream scoring.
 - Template comparison tolerates moderate speed variation.
 - Output includes per-rep timing, confidence, and normalized comparison metrics.

Story 4.4: Form score and coaching feedback generation

- **Description:** Translate kinematic metrics into user-facing coaching cues with safety-first wording.
- **Service/Owner:** FastAPI + Mobile
- **Dependencies:** Metrics pipeline, prompt templates, UI card components
- **Acceptance Criteria:**
 - User receives actionable cues tied to detected movement issues.
 - Feedback avoids medical claims and remains explicitly non-diagnostic.
 - Each cue references the relevant rep or phase when possible.
 - User can dismiss, save, or review feedback later in workout history.

Story 4.5: Ego-lifting and fatigue-risk heuristic

- **Description:** Detect combinations of velocity loss, breakdown markers, and rep quality decline to flag probable fatigue or overloaded sets.
- **Service/Owner:** FastAPI + Spring Boot
- **Dependencies:** Rep metrics, workout history, threshold tuning
- **Acceptance Criteria:**
 - Heuristic inputs and threshold versions are auditable.
 - User receives advisory feedback instead of absolute judgments.
 - Load recommendation suggestions are explainable at a high level.
 - Flagged sets do not automatically alter historical logs without user confirmation.

Epic 5: AI Coaching and Programming

Story 5.1: Conversational coaching interface

- **Description:** Build the coaching chat UI for nutrition, recovery, training, and routine modification requests.
- **Service/Owner:** Mobile + FastAPI
- **Dependencies:** Chat session model, prompt orchestration, UI state management
- **Acceptance Criteria:**
 - User can send text prompts and receive structured responses.
 - Chat supports contextual references to workouts, meals, and goals.
 - Message history is persisted and resumable.
 - AI outages return safe fallback messaging rather than blank states.

Story 5.2: Prompt assembly with user context and safety rules

- **Description:** Assemble coaching prompts using goals, profile, workout history, nutrition data, and safety disclaimers.
- **Service/Owner:** FastAPI
- **Dependencies:** Retrieval layer, profile APIs, prompt templates
- **Acceptance Criteria:**
 - Prompt assembly includes only scoped user context relevant to the request.
 - Safety instructions and product policy constraints are always attached for applicable flows.
 - Prompt versions are traceable for debugging.
 - Sensitive data exposure in prompts is minimized and logged appropriately.

Story 5.3: Personalized program generation

- **Description:** Generate structured workout programs based on goals, history, injury constraints, equipment, and schedule.
- **Service/Owner:** FastAPI + Spring Boot
- **Dependencies:** Goal setup, workout history, exercise library, prompt assembly
- **Acceptance Criteria:**
 - User can request a new plan and receive a structured weekly program draft.
 - Plan includes exercises, set/rep guidance, and progression notes.
 - Program draft can be edited before activation.
 - Activated programs are persisted as versioned plans in core backend storage.

Story 5.4: Chat-driven workout editing

- **Description:** Support chat commands for swapping exercises, converting to supersets, reducing time, and adapting to equipment constraints.
- **Service/Owner:** Mobile + FastAPI + Spring Boot
- **Dependencies:** Active plan model, exercise resolver, coaching chat
- **Acceptance Criteria:**
 - User can request common plan modifications in plain language.
 - Proposed changes are shown as a preview diff before save.
 - Invalid swaps return alternatives rather than failing silently.
 - Accepted edits update the active plan with version history.

Story 5.5: Injury triage muscle map flow

- **Description:** Create tappable muscle-map triage with guided questions and non-diagnostic coaching outputs.
- **Service/Owner:** Mobile + FastAPI
- **Dependencies:** SVG map assets, triage decision flow, safety copy
- **Acceptance Criteria:**
 - User can select a body region from an interactive map.
 - Triage flow gathers symptom context without presenting itself as diagnosis.
 - Output clearly separates fatigue guidance, cautionary language, and escalation advice.
 - High-risk responses display stronger safety guidance and suggested next steps.

Epic 6: Nutrition Goals and Daily Tracking

Story 6.1: Calorie and macro target engine

- **Description:** Calculate daily calorie and macro targets from goal, body metrics, and activity profile.
- **Service/Owner:** Spring Boot
- **Dependencies:** Profile data, formula rules, goal setup
- **Acceptance Criteria:**
 - User target calculation is reproducible from stored inputs.
 - Target engine supports maintenance, deficit, and surplus policies.
 - User can review and update target assumptions.
 - Target recalculation creates a new versioned target record rather than overwriting history.[cite:2]

Story 6.2: Micronutrient target engine

- **Description:** Add derived micronutrient targets for prioritized nutrients used in daily tracking and coaching.
- **Service/Owner:** Spring Boot
- **Dependencies:** RDA reference tables, demographics, nutrition schema
- **Acceptance Criteria:**
 - Supported micronutrients are calculated using documented rule tables.
 - Target outputs store the source version used for calculation.
 - Daily tracking can consume macro and micronutrient targets from a single target snapshot.
 - Missing demographic inputs produce defined fallback behavior.

Story 6.3: Daily nutrition dashboard and progress cards

- **Description:** Present consumed versus target values for calories, macros, and prioritized micronutrients.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Nutrition logs, target engine, UI components
- **Acceptance Criteria:**
 - Dashboard shows current day totals and remaining targets.
 - Cards refresh after meal creation, edit, or deletion.
 - Historical day views are accessible.
 - Values are consistent across dashboard, meal detail, and analytics views.[cite:2]

Story 6.4: Nutrition shortfall and coaching flags

- **Description:** Detect macro and micronutrient shortfalls or repeated deviations and surface coaching prompts.
- **Service/Owner:** Spring Boot + FastAPI
- **Dependencies:** Nutrition history, target engine, coaching rules
- **Acceptance Criteria:**
 - Rule engine can flag repeated underconsumption or overconsumption patterns.
 - Coaching flags include the metric and time window that triggered them.
 - User can dismiss or snooze suggestions.
 - Rule thresholds are configurable and versioned.

Epic 7: Scanned Meal Flow

Story 7.1: Meal photo capture and upload UX

- **Description:** Build meal capture UI for photo-based analysis with preview, retake, and submission states.
- **Service/Owner:** Mobile
- **Dependencies:** Camera permissions, media upload, analysis job API
- **Acceptance Criteria:**
 - User can capture or select a food photo.
 - Submission state communicates that analysis may take time.
 - User can cancel before upload completion.
 - Failed uploads can be retried without losing the draft meal context.

Story 7.2: Food segmentation and item proposal pipeline

- **Description:** Segment food items from an image and produce candidate dish or ingredient labels.
- **Service/Owner:** FastAPI
- **Dependencies:** Vision pipeline, upload storage, model routing
- **Acceptance Criteria:**
 - Pipeline returns one or more detected food regions when confidence is sufficient.
 - Each detected item includes a confidence score and candidate labels.
 - Low-confidence results are surfaced for manual confirmation rather than auto-accept.
 - Analysis jobs store inference metadata for later evaluation.

Story 7.3: Portion estimation and calorie-range computation

- **Description:** Estimate portion size and produce calorie and macro ranges with uncertainty handling.
- **Service/Owner:** FastAPI + Spring Boot
- **Dependencies:** Vision outputs, food composition tables, portion heuristics
- **Acceptance Criteria:**
 - Result screen displays a range rather than false precision when uncertainty is high.
 - User can manually adjust portion size before saving.
 - Final selected portion persists to the meal log.
 - Nutrition totals update immediately after confirmation.

Story 7.4: Food confirmation card and correction loop

- **Description:** Present detected food items in a conversational confirmation interface with editing controls.
- **Service/Owner:** Mobile + FastAPI + Spring Boot
- **Dependencies:** Detection pipeline, meal save API, correction model
- **Acceptance Criteria:**
 - User can confirm, rename, merge, split, or remove detected items.
 - Corrections are saved along with original detection outputs.
 - Confirmed meal entry becomes part of the daily nutrition log.
 - Correction events are usable for future model evaluation and admin review.

Story 7.5: Meal quality and timing insight generation

- **Description:** Generate food-quality cues and meal timing observations from confirmed meals and history.
- **Service/Owner:** FastAPI
- **Dependencies:** Nutrition logs, vision result metadata, coaching prompt assembly
- **Acceptance Criteria:**
 - Insights are phrased as advisory suggestions, not judgments.
 - Model outputs reference meal context and confidence boundaries.
 - User can access the explanation later from meal detail.
 - Unsafe or overreaching language is filtered before display.

Epic 8: Manual Meal Entry and Food Resolution

Story 8.1: Manual meal composer UI

- **Description:** Build manual meal entry with free-text input, structured line items, and portion fields.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Food search APIs, nutrition calculation engine
- **Acceptance Criteria:**
 - User can add a meal manually without using the camera flow.
 - Meal composer supports multiple food items per meal.
 - User can edit or remove line items before saving.
 - Saved manual entries appear in the same daily log as scanned meals.[cite:2]

Story 8.2: Indian household unit normalization

- **Description:** Normalize units such as roti, bowl, cup, katori, and spoon into standardized quantities for calculation.
- **Service/Owner:** Spring Boot
- **Dependencies:** Unit mapping table, food density heuristics, nutrition engine
- **Acceptance Criteria:**
 - Supported household units convert into defined normalized quantities.
 - Unit mappings are configurable and versioned.
 - Unsupported units trigger user clarification instead of silent miscalculation.
 - Calculation output records the normalized basis used.

Story 8.3: Food name resolution and alias matching

- **Description:** Resolve food names against the reference nutrition dataset with alias handling and ranked candidates.
- **Service/Owner:** Spring Boot
- **Dependencies:** Food database, alias tables, admin tooling
- **Acceptance Criteria:**
 - Exact and alias matches return ranked candidates.
 - Ambiguous names prompt selection from a short candidate list.
 - Resolver performance is acceptable for mobile autocomplete.
 - Selected canonical foods are used in downstream nutrition calculations.

Story 8.4: Nutrition calculation for manual entries

- **Description:** Calculate calories, macros, and supported micronutrients for manual meals.
- **Service/Owner:** Spring Boot
- **Dependencies:** Food resolution, unit normalization, target engine
- **Acceptance Criteria:**
 - Calculations update as line items or portions change.
 - Saved totals are reproducible from the underlying line items.
 - Edited meals trigger recomputation and dashboard refresh.
 - Calculation services return traceable component-level nutrition values.[cite:2]

Story 8.5: Meal editing, duplication, and templates

- **Description:** Allow users to edit prior meals, duplicate frequent meals, and save reusable meal templates.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Meal persistence, template schema
- **Acceptance Criteria:**
 - User can edit a prior meal and see updated totals.
 - User can duplicate a prior meal into the current day.
 - Frequently repeated meals can be saved as templates.
 - Template-based creation preserves editable line items rather than fixed blobs.

Epic 9: Progress, Insights, and Analytics

Story 9.1: Body metrics and check-in logging

- **Description:** Support logging of weight, measurements, photos, and subjective check-ins.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Profile schema, media storage, analytics jobs
- **Acceptance Criteria:**
 - User can log bodyweight and optional measurements.
 - Check-ins support mood, recovery, hunger, and energy markers.
 - Historical charts can query these entries by date range.
 - Photo uploads are stored securely and tied to the correct check-in.

Story 9.2: Training and nutrition correlation insights

- **Description:** Correlate nutrition adherence, sleep/recovery proxies, and workout performance trends.
- **Service/Owner:** Spring Boot + FastAPI
- **Dependencies:** Workout history, nutrition logs, analytics warehouse views
- **Acceptance Criteria:**
 - Insight engine can compute cross-domain trend summaries over defined windows.
 - Correlation messaging remains suggestive rather than falsely causal.
 - Users can inspect which data window was used.
 - Insight generation tolerates sparse data without producing misleading claims.

Story 9.3: Goal progress dashboards

- **Description:** Visualize progress toward weight, adherence, training consistency, and plan completion goals.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Check-ins, logs, active goals, analytics endpoints
- **Acceptance Criteria:**
 - Dashboard reflects current active goals.
 - Progress views can switch across week, month, and custom ranges.
 - Missing data states are clearly handled.
 - Visualizations match underlying stored aggregates.

Story 9.4: Event taxonomy and product analytics instrumentation

- **Description:** Define and implement product analytics events across onboarding, logging, coaching, and subscription flows.
- **Service/Owner:** Mobile + Spring Boot + FastAPI
- **Dependencies:** Analytics platform decision, data contract, consent handling
- **Acceptance Criteria:**
 - Core product events are named and documented in a single taxonomy.
 - Events include required IDs and timestamps without leaking unnecessary sensitive content.
 - Consent preferences are respected before analytics emission.
 - Instrumentation can support funnel and retention analysis.

Epic 10: Notifications and Engagement

Story 10.1: Workout reminders and schedule prompts

- **Description:** Send reminders for planned workouts based on user schedule and time preferences.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Push notification provider, schedule model, preferences
- **Acceptance Criteria:**
 - User can enable or disable workout reminders.
 - Reminder time windows respect the user's locale and schedule.
 - Notification tap deep-links to the appropriate app screen.
 - Duplicate reminders are prevented for the same planned session.

Story 10.2: Meal logging reminders

- **Description:** Support optional meal reminders and missed-log nudges.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Preferences, notification service, nutrition logs
- **Acceptance Criteria:**
 - User can configure meal reminder windows.
 - Reminder logic is suppressed when recent meal logs already exist.
 - Notification content avoids sensitive information on lock screen when privacy mode is enabled.
 - Delivery status is logged for debugging.

Story 10.3: Async AI result notifications

- **Description:** Notify user when long-running analyses such as meal scan or plan generation are complete.
- **Service/Owner:** Mobile + FastAPI + Spring Boot
- **Dependencies:** Background jobs, push provider, job state persistence
- **Acceptance Criteria:**
 - Long-running jobs expose statuses such as queued, running, completed, and failed.
 - User receives a notification on completion when the app is in background.
 - Tapping the notification deep-links to the completed result.
 - Failed jobs surface retry guidance.

Epic 11: Subscription, Entitlements, and Billing

Story 11.1: Plan catalog and entitlement model

- **Description:** Define free versus paid capabilities and expose entitlement checks to the app.
- **Service/Owner:** Spring Boot + Mobile
- **Dependencies:** Monetization design, feature gating policy
- **Acceptance Criteria:**
 - App can fetch active plan catalog and user entitlement state.
 - Feature gates evaluate consistently on client and server.
 - Downgraded users retain allowed historical data access.
 - Entitlement changes propagate without requiring app reinstall.

Story 11.2: In-app purchase and subscription activation flow

- **Description:** Implement subscription purchase, activation, restore, and failure handling.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Store integrations, receipt validation, entitlement model
- **Acceptance Criteria:**
 - User can purchase a subscription from the paywall.
 - Successful purchase updates entitlements promptly.
 - Restore purchase flow works on reinstall or new device.
 - Billing failures degrade access gracefully according to policy.

Story 11.3: Premium AI usage quotas and enforcement

- **Description:** Apply quota controls to AI-heavy premium features such as scan analysis and advanced coaching usage.
- **Service/Owner:** Spring Boot + FastAPI
- **Dependencies:** Entitlement model, usage metering, provider cost telemetry
- **Acceptance Criteria:**
 - Quota consumption is recorded per user and feature category.
 - Server-side enforcement prevents bypass from modified clients.
 - User sees remaining usage where relevant.
 - Quota resets and overage handling follow the configured billing cycle.

Epic 12: Offline Sync and Data Management

Story 12.1: Offline-first local cache setup

- **Description:** Implement local persistent storage for workouts, meals, plans, and pending mutations.
- **Service/Owner:** Mobile
- **Dependencies:** Local DB selection, sync protocol, schema versioning
- **Acceptance Criteria:**
 - Core user flows remain usable while offline for previously available data.
 - Mutations are stored locally until sync is available.
 - Local schema migrations preserve data across app upgrades.
 - Cache invalidation follows a defined policy.

Story 12.2: Deterministic sync protocol with conflict resolution

- **Description:** Build pull-push sync, mutation replay, and conflict handling for edits across devices.
- **Service/Owner:** Mobile + Spring Boot
- **Dependencies:** Local cache, versioned records, sync endpoints
- **Acceptance Criteria:**
 - Sync protocol can reconcile creates, updates, and deletes.
 - Conflict cases produce deterministic outcomes or user-review flows.
 - Duplicate mutation replay does not create double writes.
 - Sync status and last successful sync time are visible in the app.

Story 12.3: Media upload and object lifecycle management

- **Description:** Manage upload, retention, and access control for meal images, exercise videos, and progress photos.
- **Service/Owner:** Mobile + Spring Boot + Infra/Data
- **Dependencies:** Object storage, signed URL policy, retention rules
- **Acceptance Criteria:**
 - Media uploads are resumable or retryable on unstable networks.
 - Access to user media requires authorized URLs.
 - Retention policy can delete or archive media according to rules.
 - Media references remain consistent with parent domain records.

Epic 13: AI Orchestration and Resilience

Story 13.1: AI gateway and provider routing matrix

- **Description:** Build FastAPI gateway that routes AI tasks by type, cost tier, and fallback priority.
- **Service/Owner:** FastAPI
- **Dependencies:** Provider credentials, routing policy, request validation
- **Acceptance Criteria:**
 - AI requests are routed using a defined provider policy per task category.
 - Gateway returns normalized response contracts to callers.
 - Provider timeouts and failures can trigger fallback attempts where configured.
 - Request and response metadata are logged without leaking sensitive payloads.[cite:23]
[cite:38]

Story 13.2: Background job queue for async AI work

- **Description:** Add queue-backed execution for OCR, image analysis, plan generation, and other long-running tasks.
- **Service/Owner:** FastAPI + Infra/Data
- **Dependencies:** Queue technology, job state model, retry policy
- **Acceptance Criteria:**
 - Long-running jobs are queued instead of blocking client requests.
 - Job status can be queried by clients.
 - Retry and dead-letter behavior are defined for failed jobs.
 - Idempotency guards prevent duplicate job side effects.

Story 13.3: Embeddings and retrieval pipeline

- **Description:** Create embedding generation and retrieval workflows for relevant coaching and knowledge-grounding use cases.
- **Service/Owner:** FastAPI + Infra/Data
- **Dependencies:** pgvector, document chunking strategy, embedding provider
- **Acceptance Criteria:**
 - Supported documents and knowledge records can be embedded and indexed.
 - Retrieval queries return ranked results with source metadata.
 - Re-embedding can be triggered after content updates.
 - Index maintenance does not corrupt existing retrieval quality.

Story 13.4: Circuit breaker, retry, and provider health handling

- **Description:** Implement resilience patterns for external AI and OCR providers.
- **Service/Owner:** FastAPI
- **Dependencies:** Provider routing layer, telemetry, config service
- **Acceptance Criteria:**
 - Retries use bounded exponential backoff.
 - Consecutive provider failures can open a circuit breaker.
 - Health state influences routing decisions automatically.
 - User-facing errors degrade gracefully when all providers fail.[cite:18][cite:28]

Epic 14: Admin and Content Operations

Story 14.1: Admin console for food alias and mapping review

- **Description:** Create internal tools for reviewing unresolved foods, alias requests, and canonical mapping updates.
- **Service/Owner:** Spring Boot + Web Admin
- **Dependencies:** Role-based auth, food database, audit logs
- **Acceptance Criteria:**
 - Admins can review unresolved food names and candidate mappings.
 - Approved mappings become active without direct database edits.
 - Every mapping change is auditable by actor and timestamp.
 - Mapping updates can be rolled back if needed.

Story 14.2: Admin review queue for meal scan corrections

- **Description:** Provide internal review tools for low-confidence or high-impact meal scan corrections.
- **Service/Owner:** FastAPI + Web Admin
- **Dependencies:** Correction logging, image storage, role permissions
- **Acceptance Criteria:**
 - Internal reviewers can inspect original image, model output, and user correction.
 - Review decisions can mark outputs as accepted, corrected, or ignored.
 - Review outcomes are available to model evaluation jobs.
 - Admin actions are permissioned and audited.

Story 14.3: Prompt template and model version management

- **Description:** Manage coaching prompt templates, safety overlays, and model version rollout controls.
- **Service/Owner:** FastAPI + Web Admin
- **Dependencies:** Prompt registry, environment config, audit log
- **Acceptance Criteria:**
 - Prompt templates can be versioned and activated intentionally.
 - Rollout changes are attributable to an admin user.
 - Production prompt updates do not require direct code edits for simple text changes.
 - Rollback to a previous prompt version is supported.

Story 14.4: Exercise and coaching content management

- **Description:** Support internal editing of exercise instructions, demo assets, and coaching copy snippets.
- **Service/Owner:** Spring Boot + Web Admin
- **Dependencies:** Exercise catalog, media storage, RBAC
- **Acceptance Criteria:**
 - Internal users can edit text and media references for exercises.
 - Changes publish to mobile clients through normal API flows.
 - Content changes are versioned and auditable.
 - Invalid asset references are prevented before publish.

Epic 15: Security, Privacy, and Compliance

Story 15.1: Role-based access control and admin authorization

- **Description:** Add RBAC for end users, support users, and internal admin operations.
- **Service/Owner:** Spring Boot + FastAPI
- **Dependencies:** Auth, permission model, admin UI
- **Acceptance Criteria:**
 - Protected endpoints enforce role and permission checks.
 - Admin-only features are inaccessible to normal users.
 - Permission failures produce safe responses without leaking internal detail.
 - Permission changes are auditable.

Story 15.2: Encryption and secret management baseline

- **Description:** Establish encryption in transit, secure secret storage, and key rotation processes.
- **Service/Owner:** Infra/Data
- **Dependencies:** Deployment platform, secret manager, TLS setup
- **Acceptance Criteria:**
 - All external traffic uses TLS.
 - Sensitive credentials are not stored in source control or plaintext configs.
 - Secret rotation procedure is documented and tested.
 - Storage of sensitive user data follows defined encryption rules.[cite:28][cite:35]

Story 15.3: Data export, account deletion, and retention controls

- **Description:** Allow users to export data, request deletion, and manage retention-sensitive content.
- **Service/Owner:** Spring Boot + Mobile + Infra/Data
- **Dependencies:** Data inventory, legal retention policy, background jobs
- **Acceptance Criteria:**
 - User can request an account export in a documented format.
 - User can request account deletion from settings.
 - Deletion workflow handles domain records and media consistently.
 - Retention exceptions are logged and policy-driven.

Story 15.4: Sensitive content handling and privacy-aware notifications

- **Description:** Prevent accidental disclosure of health-related context in notifications, logs, and analytics.
- **Service/Owner:** Mobile + Spring Boot + FastAPI
- **Dependencies:** Notification service, analytics taxonomy, logging policy
- **Acceptance Criteria:**
 - Privacy mode suppresses sensitive message content on lock screen.
 - Logs avoid storing raw sensitive inputs unless explicitly required and controlled.
 - Analytics events exclude disallowed content fields.
 - Notification templates are reviewed against privacy rules.

Epic 16: Observability, QA, and Release Engineering

Story 16.1: Structured logging, metrics, and distributed tracing

- **Description:** Add observability across Spring Boot, FastAPI, jobs, and critical mobile flows.
- **Service/Owner:** Infra/Data + Spring Boot + FastAPI
- **Dependencies:** Telemetry stack, correlation IDs, deployment config
- **Acceptance Criteria:**
 - Services emit structured logs with correlation IDs.
 - API latency, error, and provider health metrics are available on dashboards.
 - Background job execution can be traced across services.
 - Sensitive data is redacted from telemetry by policy.[cite:28][cite:35]

Story 16.2: Crash reporting and mobile performance monitoring

- **Description:** Integrate mobile crash monitoring and performance diagnostics.
- **Service/Owner:** Mobile
- **Dependencies:** Monitoring SDK, release metadata, privacy review
- **Acceptance Criteria:**
 - Crashes are grouped by release version and device metadata.
 - Startup time and key screen performance are measurable.
 - Monitoring respects privacy and consent requirements.
 - Release regressions are visible in dashboards.

Story 16.3: Automated API, contract, and E2E testing

- **Description:** Create automated tests for domain services, AI gateway contracts, and critical mobile user journeys.
- **Service/Owner:** Spring Boot + FastAPI + Mobile
- **Dependencies:** Test frameworks, seeded data, CI pipeline
- **Acceptance Criteria:**
 - Core API contracts are covered by automated tests.
 - AI gateway response normalization has contract coverage.
 - Critical mobile flows have end-to-end coverage.
 - CI fails on regression in required test suites.[cite:35][cite:36]

Story 16.4: CI/CD pipeline and environment promotion

- **Description:** Set up build, test, image publish, deployment, and promotion workflows for all services.
- **Service/Owner:** Infra/Data
- **Dependencies:** Repository structure, secret management, deployment targets
- **Acceptance Criteria:**
 - Every service builds and tests automatically on pull requests.
 - Versioned artifacts are published for deployable services.
 - Staging and production promotions follow a defined workflow.
 - Rollback procedures exist and are tested.

Story 16.5: Release gating, feature flags, and staged rollout

- **Description:** Introduce feature flags and release controls for high-risk features such as meal scan and advanced coaching.
- **Service/Owner:** Spring Boot + FastAPI + Mobile
- **Dependencies:** Config service, analytics events, deployment process
- **Acceptance Criteria:**
 - High-risk features can be enabled or disabled without full redeploy.
 - Rollout can be limited by environment or cohort.
 - Feature-flag state is visible to support and engineering.
 - Rollback from a bad rollout can be executed quickly.

Plane setup recommendations

Use each H2 epic as a Plane epic and create each H3 story as a child issue under it. This aligns with agile guidance that epics should be broken into smaller pieces of work and refined into testable stories before implementation.[cite:24][cite:26][cite:27]

Suggested labels for Plane:

- mobile
- spring-boot
- fastapi
- infra
- analytics
- security
- billing
- admin

- ai-orchestration
- nutrition
- workouts

Suggested custom fields:

- Service Owner
- Priority
- Release Bucket
- Risk Level
- Blocked By
- Acceptance Criteria Complete

Implementation notes

This backlog intentionally goes beyond a feature inventory and includes the operational and non-functional work needed for a production system. Non-functional requirements such as resilience, performance, security, and testability should remain visible in the backlog and in the definition of done for relevant stories rather than being treated as afterthoughts.[cite:18][cite:28][cite:33][cite:35]

The backlog should still be refined further during execution, especially before stories are committed to a sprint. Best practice is to keep higher-priority items smaller, testable, and implementation-ready while leaving lower-priority items at a broader level until they move closer to delivery.[cite:24][cite:26][cite:27][cite:30][cite:35]